

General objectives of the exercise

The request is to build a full-stack application (backend API + frontend SPA) to manage campaigns and their associated creatives.

Result

The goal is to assess:

- API design
- Data modelling
- Frontend integration
- Engineering quality

Campaigns are provided via a file (JSON or CSV).

Creatives are not provided and must be implemented.

No database setup is required.

Each campaign includes:

- id
- name
- status (1 = Active, 0 = Paused)
- landingUrl
- coverImageUrl
- createdAt

Each creative must include:

- id (generated server-side)
- campaignId
- assetUrl (320×480)
- createdAt (generated server-side)

Business Rules

- A campaign with status = 0 (Paused) cannot accept new creatives.
- A campaign can have at most 3 creatives associated with it.

These rules must be enforced at the backend level.

Part 1 – Backend Development

Allowed Technologies:

- Preferred: Node.js (Express or similar)
- Alternative: PHP (CakePHP / Laravel / Symfony)
- Serverless (AWS Lambda + API Gateway) is welcome but not required

Required Endpoints:

- GET /api/campaigns (pagination + filters: status, q, page, limit)
- GET /api/campaigns/:id
- PUT /api/campaigns/:id (id required in body; status must be 0 or 1; landingUrl valid if provided)
- GET /api/campaigns/:id/creatives
- POST /api/campaigns/:id/creatives (image must be 320×480; file upload or base64 allowed)

Part 2 – Frontend Development

Technical Constraints:

- JavaScript (ES6+)
- React (preferred) / Vue / Angular / Backbone
- Basic styling is fine
- Optional UI Libraries: Material Design Lite, Material UI (React), Semantic UI, Semantic UI React, Bootstrap

Frontend Requirements:

- Use the view system of your chosen framework
- Perform client-side API calls to your backend
- Handle loading state, empty state, and error state

Required Features:

- Display campaigns

- Filter by id, name, and status (Active/Paused)
- Edit campaign (name, status, landingUrl, imageUrl)
- View creatives per campaign
- Upload and create creatives (320×480)
- Favourite campaigns stored in localStorage

Nice to have

- TypeScript
- Unit tests
- Accessibility best practices
- If localStorage is not supported, use cookies as fallback
- A short README section explaining performance considerations and how you would productionise this

Suggestions

- We prefer you to run both backend and frontend parts rather than completing only one perfectly.
- Once you choose a framework, stick to its specifications and official documentation.
- Write abstract and reusable functions, foreseeing possible future growth of the application.
- Keep the solution simple and clean.
- Avoid overengineering.
- Pay attention to performance.
- Once you have chosen the UI framework, explore its available components (buttons, layouts, etc.) and use those that best fit your layout.